- api_endpoint.py — The REST Gateway to the Agentic AI Analysis System
  - Table of Contents
  - 1. What Does This File Do?
  - 2. Where Does It Sit in the Architecture?
  - 3. High-Level System Flow
  - 4. The Request Lifecycle — Step by Step
    - Step 1: Frontend sends POST
    - Step 2: api_endpoint.py creates tracking entry
    - Step 3: Background task starts
    - Step 4: Background connects to MCP Server
    - Step 5: Set per-analysis output directory
    - Step 6: Fetch tools and build the agent graph
    - Step 7: Run the multi-agent pipeline
    - Step 8: Store results
    - Step 9: Frontend picks up results
  - 5. Code Walkthrough
    - 5.1 Imports & Setup
    - 5.2 Data Models (Pydantic)
    - 5.3 Endpoints
      - POST /analyze — Start an Analysis
      - GET /status/{analysis_id} — Poll for Progress
      - GET /reports — List All Reports Across All Analyses
      - GET /reports/{analysis_id}/{filename} — Download a File
      - DELETE /analysis/{analysis_id} — Delete an Analysis
    - 5.4 Background Runner: _run_analysis_background
  - 6. How It Connects to Other Files
    - Connection Map
    - File-by-File Relationship
  - 7. Output Isolation — Per-Analysis Subfolders
    - How isolation works:
  - 8. Polling Protocol — Frontend ↔ Backend
  - 9. Error Handling
    - API Level (Endpoint Errors)
    - Background Task Level (Pipeline Errors)
  - 10. Key Concepts Explained
    - What is MCP?
    - What is LangGraph?

    - Why asyncio.create_task()?
    - Why _analyses is a dict (in-memory)?
  - 11. Quick Reference: All Endpoints
  - Summary


# `api_endpoint.py` — The REST Gateway to the Agentic AI Analysis System

> **File:** `python/agentic/api_endpoint.py` **Role:** FastAPI router that exposes the multi-agent LangGraph pipeline as a REST API for the frontend UI.

---

## Table of Contents


1. What Does This File Do?
2. Where Does It Sit in the Architecture?
3. High-Level System Flow
4. The Request Lifecycle — Step by Step
5. Code Walkthrough
    - Imports & Setup
    - Data Models
    - Endpoints
    - Background Runner
6. How It Connects to Other Files
7. Output Isolation — Per-Analysis Subfolders
8. Polling Protocol — Frontend ↔ Backend
9. Error Handling
10. Key Concepts Explained
11. Quick Reference: All Endpoints


---

## 1. What Does This File Do?

`api_endpoint.py` is the **bridge between the web UI and the AI analysis engine**. It:

- Accepts analysis questions from the frontend (e.g., *"Compare ore feed rates across all mills for the last 72 hours"*)
- Kicks off a **background** multi-agent pipeline (LangGraph + MCP tools)
- Lets the frontend **poll** for progress and results
- Serves generated **charts** (.png) and **reports** (.md) back to the UI
- Manages **cleanup** when users delete conversations

Think of it as a **reception desk**: it takes your request, hands it to the AI team working in the back room, and lets you check on progress or pick up results.

---

# 2. Where Does It Sit in the Architecture?

```
┌──────────────────────────────────────────────────────────────────────┐
│                        FRONTEND (Next.js)                            │
│                                                                      │
│   ┌──────────────┐    ┌──────────────┐    ┌──────────────────┐      │
│   │  AI Chat UI  │──>│  Chat Store  │──>│  Next.js Proxy   │      │
│   │  (page.tsx)  │<──│  (Zustand)   │<──│  /api/agentic/*  │      │
│   └──────────────┘    └──────────────┘    └──────────┬───────┘      │
│                                                       │              │
└──────────────────────────────────────────────────────┼──────────────┘
                                                       │ HTTP
┌──────────────────────────────────────────────────────┼──────────────┐
│                     BACKEND (FastAPI)                │              │
│                                                      ▼              │
│   ┌──────────┐    ┌──────────────────────────────────┐             │
│   │  api.py  │──>│  api_endpoint.py  ◄── THIS FILE  │             │
│   │  (main)  │    │  (APIRouter at /api/v1/agentic)  │             │
│   └──────────┘    └──────────────┬──────────────────┘             │
│                                  │                                  │
│                                  │ asyncio.create_task()            │
│                                  ▼                                  │
│                    ┌──────────────────────┐                         │
│                    │ _run_analysis_bg()   │                         │
│                    │  (background task)   │                         │
│                    └──────────┬───────────┘                         │
│                               │                                     │
│              ┌────────────────┼────────────────┐                    │
│              ▼                ▼                ▼                    │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐          │
│   │  client.py   │   │  graph_v3.py │   │  output/     │          │
│   │ (MCP bridge) │   │ (LangGraph)  │   │  {id}/       │          │
│   └──────┬───────┘   └──────────────┘   │  ├─ chart.png │          │
│          │                               │  └─ report.md │          │
│          │ MCP Protocol (HTTP)           └──────────────┘          │
│          ▼                                                          │
│   ┌──────────────────────────────────┐                              │
│   │  server.py (MCP Server, port 8003)  │                           │
```

```
│   │  ┌──────────────────────────────┐  │                          │
│   │  │          tools/              │  │                          │
│   │  │  ├─ db_tools.py      (SQL)   │  │                          │
│   │  │  ├─ python_executor.py (code) │  │                          │
│   │  │  ├─ report_tools.py  (files)  │  │                          │
│   │  │  └─ session_tools.py (config) │  │                          │
│   │  └──────────────────────────────┘  │                          │
│   └────────────────────────────────────┘                          │
│                      │                                             │
│                      ▼                                             │
│              ┌──────────────────┐                                  │
│              │   PostgreSQL DB  │                                  │
│              │  (Mill Data +    │                                  │
│              │   Ore Quality)   │                                  │
│              └──────────────────┘                                  │
└────────────────────────────────────────────────────────────────────┘
```

**Key insight:** `api_endpoint.py` doesn't do any analysis itself. It **orchestrates** — it connects the UI to the LangGraph multi-agent system via an MCP client connection.

---

# 3. High-Level System Flow

```
User types question in AI Chat
           │
           ▼
┌──────────────────────────────┐
│  POST /api/v1/agentic/analyze│  ◄── api_endpoint.py receives request
│  Returns: { analysis_id }    │
└──────────────────────────────┘
              │
              │
   ┌──────────┴──────────┐
   │  Frontend starts    │
   │  polling every 4s   │
   │                     │
   ▼                     ▼
┌──────────────┐   ┌──────────────────────────────────────────┐
│ GET /status  │   │  BACKGROUND TASK (runs 2-5 minutes)      │
│ returns      │   │                                          │
│ "running"    │   │  1. Connect to MCP Server (port 8003)    │
│              │   │  2. Set output dir → output/{analysis_id}/ │
│              │   │  3. Fetch MCP tools → wrap as LangChain tools│
│              │   │  4. Build LangGraph (planner + 6 specialists) │
│              │   │  5. Run graph with user's question       │
│              │   │     ┌──────────────────────────────────┐ │
│              │   │     │ data_loader → planner            │ │
│              │   │     │ planner selects 1-4 specialists from:│ │
│              │   │     │   analyst, forecaster, anomaly_det, │ │
│              │   │     │   bayesian, optimizer, shift_reporter │ │
│              │   │     │ each specialist → manager_review  │ │
```

```
│              │    │     │ code_reviewer → reporter → END        │ │
│              │    │     └──────────────────────────────────────┘ │
│              │    │  6. Save final answer + update status        │
│              │    └──────────────────────────────────────────────┘
│              │
│   (after completion)
│              ▼
│   ┌──────────────────────┐
│   │ GET /status returns   │
│   │ "completed" +         │
│   │ final_answer +        │
│   │ chart_files +         │
│   │ report_files          │
│   └──────────────────────┘
│              │
│              ▼
│   ┌──────────────────────────┐
│   │ GET /reports/{id}/{filename} │  ◄── Serve charts & reports
│   └──────────────────────────┘
└──────────────────────────────────
```

---

# 4. The Request Lifecycle — Step by Step

Here's exactly what happens when a user asks: *"Сравни средното натоварване по руда на всички мелници за последните 72 часа"*

## Step 1: Frontend sends POST

```
POST /api/v1/agentic/analyze
Body: { "question": "Сравни средното натоварване...", "mill_number": null }
```

## Step 2: `api_endpoint.py` creates tracking entry

```
analysis_id = "a3f7b2c1"  # random 8-char ID

_analyses["a3f7b2c1"] = {
    "status": "running",
    "question": "Сравни средното натоварване...",
    "started_at": "2026-03-26T11:00:00",
    "final_answer": None,
```

```
    "error": None,
    "completed_at": None,
}
```

## Step 3: Background task starts

```
asyncio.create_task(_run_analysis_background("a3f7b2c1", full_prompt))
```

The endpoint **returns immediately** with the `analysis_id`. The heavy work runs in the background.

## Step 4: Background connects to MCP Server

```
async with streamable_http_client("http://localhost:8003/mcp") as (read, write, _):
    async with ClientSession(read, write) as session:
        await session.initialize()
```

## Step 5: Set per-analysis output directory

```
await session.call_tool("set_output_directory", {"analysis_id": "a3f7b2c1"})
# → All charts/reports now go to: output/a3f7b2c1/
```

## Step 6: Fetch tools and build the agent graph

```
langchain_tools = await get_mcp_tools(session)  # 7 tools from MCP server
graph = build_graph(langchain_tools, api_key)    # LangGraph with planner + 6
specialist pool
```

## Step 7: Run the multi-agent pipeline

```
final_state = await graph.ainvoke(
    {"messages": [HumanMessage(content=prompt)]},
    config={"configurable": {"thread_id": "a3f7b2c1"}, "recursion_limit": 150},
)
```

The graph executes a **dynamic pipeline** (graph_v3 planner-driven architecture):

```
data_loader → [loads SQL data] → manager_review → ✓
    planner → [selects specialists] → manager_review → ✓
    specialist₁ → [e.g. analyst: EDA, SPC, charts] → manager_review → ✓
    specialist₂ → [e.g. forecaster: Prophet, ARIMA] → manager_review → ✓
    ...up to 4 specialists chosen by planner...
    code_reviewer → [validates all outputs] → manager_review → ✓
    reporter → [writes markdown report] → END
```

The planner dynamically selects from: `analyst`, `forecaster`, `anomaly_detective`, `bayesian_analyst`, `optimizer`, `shift_reporter`.

## Step 8: Store results

```
_analyses["a3f7b2c1"]["status"] = "completed"
_analyses["a3f7b2c1"]["final_answer"] = final_state["messages"][-1].content
```

## Step 9: Frontend picks up results

```
GET /api/v1/agentic/status/a3f7b2c1
→ { status: "completed", final_answer: "...", chart_files: ["ore_comparison.png"],
... }

GET /api/v1/agentic/reports/a3f7b2c1/ore_comparison.png
→ [PNG binary file]
```

---

# 5. Code Walkthrough

## 5.1 Imports & Setup

```
from client import get_mcp_tools       # Fetches tools from MCP server → LangChain
wrappers
from graph_v3 import build_graph       # Builds the planner-driven multi-agent
LangGraph pipeline

router = APIRouter(prefix="/api/v1/agentic", tags=["agentic"])

OUTPUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "output")
SERVER_URL = "http://localhost:8003/mcp"

_analyses: dict[str, dict] = {}  # In-memory tracking for running/completed
analyses
```

| Variable | Purpose |
|---|---|
| `router` | FastAPI router — all endpoints have prefix `/api/v1/agentic` |
| `OUTPUT_DIR` | Base directory for analysis outputs (`agentic/output/`) |
| `SERVER_URL` | MCP Server address — must be running on port 8003 |
| `_analyses` | Dictionary tracking all analysis jobs by their ID |

## 5.2 Data Models (Pydantic)

Three Pydantic models define the API contract:

```
┌──────────────────────────┐
│   AnalysisRequest        │  ◄── What the frontend sends
│  ┌────────────────────┐  │
│  │ question: str      │  │  "Compare ore rates for all mills"
│  │ mill_number?: int  │  │  Optional: focus on specific mill
│  │ start_date?: str   │  │  Optional: ISO date range start
│  │ end_date?: str     │  │  Optional: ISO date range end
│  └────────────────────┘  │
└──────────────────────────┘


┌──────────────────────────┐
│   AnalysisResponse       │  ◄── Immediate response after POST
│  ┌────────────────────┐  │
│  │ analysis_id: str   │  │  "a3f7b2c1" — use this to poll
│  │ status: str        │  │  "running"
│  │ message: str       │  │  "Use GET /status/{id} to check"
│  │ started_at: str    │  │  ISO timestamp
│  └────────────────────┘  │
└──────────────────────────┘


┌──────────────────────────┐
│   AnalysisResult         │  ◄── Full result from GET /status
```

```
│  ┌──────────────────┐ │
│  │ analysis_id: str  │ │
│  │ status: str       │ │  "running" | "completed" | "failed"
│  │ question: str     │ │
│  │ final_answer?: str │ │  The AI's complete answer
│  │ report_files: []  │ │  ["mill_analysis.md"]
│  │ chart_files: []   │ │  ["ore_comparison.png", "spc_chart.png"]
│  │ started_at: str   │ │
│  │ completed_at?: str │ │
│  │ error?: str       │ │  Error message if failed
│  └──────────────────┘ │
└──────────────────────┘
```

## 5.3 Endpoints

### `POST /analyze` — Start an Analysis

```
@router.post("/analyze", response_model=AnalysisResponse)
async def start_analysis(request: AnalysisRequest):
```

**What it does:**

1. Generates a short unique ID: `str(uuid.uuid4())[:8]`
2. Builds a full prompt by combining `question` + optional `mill_number`, `start_date`, `end_date`
3. Creates a tracking entry in `_analyses` with status `"running"`
4. Fires off `_run_analysis_background()` as an **async background task**
5. Returns immediately with the `analysis_id`

**Why background?** The agent pipeline takes 2-5 minutes. We can't hold the HTTP connection open that long. Instead, the frontend polls.

---

### `GET /status/{analysis_id}` — Poll for Progress

```
@router.get("/status/{analysis_id}", response_model=AnalysisResult)
async def get_analysis_status(analysis_id: str):
```

**What it does:**

1. Looks up `analysis_id` in `_analyses` — returns 404 if not found

2. Scans the analysis output directory (`output/{analysis_id}/`) for files
3. Categorizes files: `.md` → `report_files`, `.png` → `chart_files`
4. Returns the full `AnalysisResult` with current status, answer, and file lists

**Called by the frontend every ~4 seconds** until `status` is `"completed"` or `"failed"`.

---

### `GET /reports` — List All Reports Across All Analyses

```
@router.get("/reports")
async def list_reports():
```

Returns a summary of **all** generated files across all analysis subfolders. Useful for browsing historical outputs.

---

### `GET /reports/{analysis_id}/{filename}` — Download a File

```
@router.get("/reports/{analysis_id}/{filename}")
async def get_report_file(analysis_id: str, filename: str):
```

Serves a specific chart or report file. Uses `FileResponse` for efficient file streaming. Includes a **fallback** to the flat `output/` directory for backward compatibility with older analyses.

---

### `DELETE /analysis/{analysis_id}` — Delete an Analysis

```
@router.delete("/analysis/{analysis_id}")
async def delete_analysis(analysis_id: str):
```

**What it does:**

1. Removes the output subfolder (`output/{analysis_id}/`) with `shutil.rmtree`
2. Removes the in-memory tracking entry from `_analyses`

Called by the frontend when a user deletes a conversation.

---

# 5.4 Background Runner: `_run_analysis_background`

This is the **heart** of the file — the function that actually runs the AI pipeline:

```
async def _run_analysis_background(analysis_id: str, prompt: str) -> None:
```

Here's the detailed flow:

```
_run_analysis_background("a3f7b2c1", "Compare ore rates...")
    │
    ├─ 1. Get GOOGLE_API_KEY from environment
    │
    ├─ 2. Open MCP connection to server.py (port 8003)
    │     └─ streamable_http_client("http://localhost:8003/mcp")
    │
    ├─ 3. Initialize MCP session
    │     └─ session.initialize()
    │
    ├─ 4. Set output directory for this analysis
    │     └─ session.call_tool("set_output_directory", {"analysis_id": "a3f7b2c1"})
    │     └─ All files now go to: output/a3f7b2c1/
    │
    ├─ 5. Fetch all 7 MCP tools and wrap as LangChain tools
    │     └─ langchain_tools = await get_mcp_tools(session)
    │         ├─ get_db_schema        → inspect database tables
    │         ├─ query_mill_data      → load mill sensor data
    │         ├─ query_combined_data  → load mill + ore quality data
    │         ├─ execute_python       → run analysis code (pandas, matplotlib)
    │         ├─ list_output_files    → check generated charts
    │         ├─ write_markdown_report→ write final report
    │         └─ set_output_directory → configure output path
    │
    ├─ 6. Build LangGraph with planner + 6 specialist pool
    │     └─ graph = build_graph(langchain_tools, api_key)
    │         ├─ data_loader       → uses: query_mill_data, query_combined_data,
get_db_schema
    │         ├─ planner           → no tools (text-only, selects specialists)
    │         ├─ analyst           → uses: execute_python, list_output_files
    │         ├─ forecaster        → uses: execute_python, list_output_files
    │         ├─ anomaly_detective → uses: execute_python, list_output_files
    │         ├─ bayesian_analyst  → uses: execute_python, list_output_files
    │         ├─ optimizer         → uses: execute_python, list_output_files
    │         ├─ shift_reporter    → uses: execute_python, list_output_files
    │         ├─ code_reviewer     → uses: execute_python, list_output_files
    │         └─ reporter          → uses: list_output_files, write_markdown_report
    │
    ├─ 7. Run the graph
    │     └─ final_state = await graph.ainvoke(...)
    │         ├─ Thread ID = analysis_id (for state isolation)
```

```
    │         └─ Recursion limit = 150 (max LangGraph steps, increased for dynamic
pipeline)
    │
    ├─ 8. On SUCCESS:
    │     ├─ _analyses[id]["status"] = "completed"
    │     ├─ _analyses[id]["final_answer"] = last message content
    │     └─ _analyses[id]["completed_at"] = timestamp
    │
    └─ 9. On FAILURE:
          ├─ _analyses[id]["status"] = "failed"
          ├─ _analyses[id]["error"] = error message
          └─ _analyses[id]["completed_at"] = timestamp
```

---

# 6. How It Connects to Other Files

## Connection Map

```
api_endpoint.py
    │
    ├── imports ──► client.py
    │               │
    │               │  get_mcp_tools(session)
    │               │  - Connects to MCP server
    │               │  - Fetches tool descriptors
    │               │  - Wraps each as LangChain StructuredTool
    │               │  - JSON Schema → Pydantic model conversion
    │               │
    ├── imports ──► graph_v3.py
    │               │
    │               │  build_graph(tools, api_key)
    │               │  - Creates planner + 6-specialist-pool architecture
    │               │  - Planner dynamically selects 1-4 specialists per request
    │               │  - Binds specific tools to each agent
    │               │  - Wires up the LangGraph state machine with manager review
gates
    │               │  - Returns compiled graph ready for .ainvoke()
    │               │
    ├── mounted by ── api.py
    │               │
    │               │  Main FastAPI app imports this router:
    │               │    from api_endpoint import router as agentic_router
    │               │    app.include_router(agentic_router)
    │               │  Routes become available at /api/v1/agentic/*
    │               │
    ├── connects to ── server.py (via MCP protocol, port 8003)
    │               │
    │               │  MCP Server hosts the tool registry:
    │               │    tools/__init__.py defines 7 tools
    │               │    server.py exposes them via StreamableHTTP
```

```
      │                │
      │── writes to ──► output/{analysis_id}/
                       │
                       │  Charts (.png) and reports (.md)
                       │  Created by execute_python and write_markdown_report tools
```

# File-by-File Relationship

| File | Role | How `api_endpoint.py` Uses It |
|---|---|---|
| `api.py` | Main FastAPI app | Mounts `api_endpoint.py`'s router at startup |
| `client.py` | MCP → LangChain bridge | `get_mcp_tools()` converts MCP tools to LangChain tools |
| `graph_v3.py` | Multi-agent pipeline | `build_graph()` creates planner-driven 6-specialist LangGraph |
| `server.py` | MCP Server (port 8003) | Hosts the tools that agents call during analysis |
| `tools/__init__.py` | Tool registry | Defines the 7 available MCP tools |
| `tools/db_tools.py` | Database access | `query_mill_data`, `query_combined_data`, `get_db_schema` |
| `tools/python_executor.py` | Code execution | `execute_python` — runs pandas/matplotlib code |
| `tools/report_tools.py` | File management | `list_output_files`, `write_markdown_report` |
| `tools/session_tools.py` | Session config | `set_output_directory` — per-analysis isolation |
| `tools/output_dir.py` | Shared state | Manages the current output directory path |

# 7. Output Isolation — Per-Analysis Subfolders

Each analysis writes to its own isolated directory to prevent file collisions:

```
output/
├── a3f7b2c1/                  ←── Analysis 1
│   ├── ore_comparison.png
│   ├── spc_chart_psi80.png
│   └── mill_analysis.md
├── b7e2d4f9/                  ←── Analysis 2
│   ├── distribution_plot.png
│   └── quick_report.md
└── c1a9e3b5/                  ←── Analysis 3
    ├── correlation_heatmap.png
    ├── anomaly_detection.png
    └── full_report.md
```

## How isolation works:

```
api_endpoint.py                           server.py (MCP)
      │                                         │
      │  session.call_tool(                     │
      │    "set_output_directory",              │
      │    {"analysis_id": "a3f7b2c1"}         │
      │  )                                      │
      │──────── MCP call ──────────────────────►│
      │                                         │
      │                                   tools/session_tools.py
      │                                         │
      │                                   set_output_dir("a3f7b2c1")
      │                                         │
      │                                   tools/output_dir.py
      │                                         │
      │                                   _current_output_dir =
      │                                     "agentic/output/a3f7b2c1"
      │                                         │
      │  (Now all tools use get_output_dir()    │
      │   which returns the subfolder path)      │
```

When the user deletes a conversation, `DELETE /analysis/{id}` removes the entire subfolder.

---

# 8. Polling Protocol — Frontend ↔ Backend

The frontend uses a **simple polling** pattern (not WebSocket/SSE) because agent runs take 2-5 minutes:

```
Frontend (chat-store.ts)                    Backend (api_endpoint.py)
         |                                          |
  t=0s   |  POST /analyze                           |
         |  { question: "..." }                     |
         |──────────────────────────────────────────►|
         |                                          | Creates task
         |◄──────────────────────────────────────────|
         |  { analysis_id: "a3f7b2c1",              |
         |    status: "running" }                    |
         |                                          |
  t=4s   |  GET /status/a3f7b2c1                    |
         |──────────────────────────────────────────►|
         |◄──────────────────────────────────────────|
         |  { status: "running", chart_files: [] }  |
         |                                          |
  t=8s   |  GET /status/a3f7b2c1                    |
         |──────────────────────────────────────────►|
         |◄──────────────────────────────────────────|
         |  { status: "running",                    |
         |    chart_files: ["ore_comp.png"] }        | ◄── Chart created!
         |                                          |
  ...    |  (continues polling every 4s)            |
         |                                          |
  t=180s |  GET /status/a3f7b2c1                    |
         |──────────────────────────────────────────►|
         |◄──────────────────────────────────────────|
         |  { status: "completed",                  |
         |    final_answer: "## Analysis Report...", |
         |    chart_files: ["ore_comp.png", ...],   |
         |    report_files: ["report.md"] }          |
         |                                          |
         |  (stops polling)                         |
         |                                          |
         |  GET /reports/a3f7b2c1/ore_comp.png      |
         |──────────────────────────────────────────►|
         |◄──────────────────────────────────────────|
         |  [PNG binary data]                       |
```

---

# 9. Error Handling

The system handles errors at two levels:

## API Level (Endpoint Errors)

```
# Unknown analysis ID → 404
if analysis_id not in _analyses:
    raise HTTPException(status_code=404, detail=f"Analysis {analysis_id} not
found")

# Missing file → 404
if not os.path.exists(file_path):
    raise HTTPException(status_code=404, detail=f"File {filename} not found")
```

## Background Task Level (Pipeline Errors)

```
try:
    # ... entire pipeline ...
    _analyses[analysis_id]["status"] = "completed"
except Exception as e:
    _analyses[analysis_id]["status"] = "failed"
    _analyses[analysis_id]["error"] = str(e)
```

Any failure in the pipeline — missing API key, MCP server down, LLM error, tool crash — gets caught and stored. The frontend sees `status: "failed"` with the error message on the next poll.

---

# 10. Key Concepts Explained

## What is MCP?

**Model Context Protocol** — a standard for giving AI models access to external tools.

```
┌──────────────┐      MCP Protocol      ┌──────────────┐
│  MCP Client  │ ←──────────────────→ │  MCP Server  │
│  (client.py) │     (HTTP/JSON)        │ (server.py)  │
│              │                         │              │
│ "I need to   │  list_tools()          │ "Here are    │
```

```
│  query data" │ ──────────────►       │  7 tools"    │
│              │                        │              │
│              │  call_tool(            │ "Running     │
│              │   "query_mill_data",   │  SQL query..."│
│              │   {mill: 8})           │              │
│              │ ──────────────►       │              │
│              │  ◄──────────────       │ "Loaded 5000 │
│              │  result: {...}         │  rows"       │
└──────────────┘                        └──────────────┘
```

# What is LangGraph?

**LangGraph** is a framework for building stateful, multi-agent AI workflows as directed graphs.

```
  graph_v3.py builds this (planner-driven dynamic pipeline):

  ┌──────────────┐      ┌─────────┐
  │  data_loader  │◄──►│ tools  │  (query_mill_data, query_combined_data,
get_db_schema)
  └──────────────┘      └─────────┘
         │
         ▼
  ┌──────────────┐
  │ manager_review  │  (auto-ACCEPT for data_loader)
  └──────────────┘
         │
         ▼
  ┌──────────────┐
  │    planner    │  (no tools — selects 1-4 specialists from pool of 6)
  └──────────────┘
         │
         ▼
  ┌──────────────┐
  │ manager_review  │  (auto-ACCEPT for planner)
  └──────────────┘
         │
         ▼ (dynamic: 1-4 specialists chosen by planner)
  ┌──────────────┐      ┌─────────┐
  │  specialist₁  │◄──►│ tools  │  (execute_python, list_output_files)
  │  (e.g. analyst)  │      └─────────┘
  └──────────────┘
         │
         ▼
  ┌──────────────┐
  │ manager_review  │── REWORK? ──► back to specialist (max 1 rework)
  └──────────────┘
         │ ACCEPT
         ▼
  ┌──────────────┐      ┌─────────┐
  │  specialist₂  │◄──►│ tools  │  (execute_python, list_output_files)
  │  (e.g. forecaster)│      └─────────┘
  └──────────────┘
         │
         ▼
  ┌──────────────┐
```

```
    │ manager_review  │──── REWORK? ──► back to specialist
    └─────────────────┘
            │ ACCEPT
            ▼  ...more specialists if selected...
    ┌─────────────────┐     ┌─────────┐
    │ code_reviewer   │◄───►│ tools   │  (execute_python, list_output_files)
    └─────────────────┘     └─────────┘
            │
            ▼
    ┌─────────────────┐
    │ manager_review  │──── REWORK? ──► back to code_reviewer
    └─────────────────┘
            │ ACCEPT
            ▼
    ┌─────────────────┐     ┌─────────┐
    │   reporter      │◄───►│ tools   │  (list_output_files, write_markdown_report)
    └─────────────────┘     └─────────┘
            │
            ▼
    ┌─────────────────┐
    │ manager_review  │  ACCEPT → END
    └─────────────────┘

  Specialist pool: analyst, forecaster, anomaly_detective,
                   bayesian_analyst, optimizer, shift_reporter
```

# Why `asyncio.create_task()`?

This is Python's way of saying *"start this function in the background and don't wait for it."*

```
# WITHOUT create_task — blocks for 3+ minutes! ❌
await _run_analysis_background(analysis_id, prompt)
return AnalysisResponse(...)  # User waits 3+ min for this!

# WITH create_task — returns immediately ✅
asyncio.create_task(_run_analysis_background(analysis_id, prompt))
return AnalysisResponse(...)  # User gets response instantly
```

# Why `_analyses` is a dict (in-memory)?

```
_analyses: dict[str, dict] = {}
```

This is a simple in-memory dictionary. It means:

- **Fast** — no database overhead for status checks

- **Ephemeral** — data is lost if the server restarts
- **Good enough** — analyses typically complete in minutes, not days

For production, this could be replaced with Redis or a database table.

---

# 11. Quick Reference: All Endpoints

| Method | Path | Purpose | Returns |
|---|---|---|---|
| `POST` | `/api/v1/agentic/analyze` | Start a new analysis | `AnalysisResponse` with `analysis_id` |
| `GET` | `/api/v1/agentic/status/{id}` | Poll analysis progress | `AnalysisResult` with status + files |
| `GET` | `/api/v1/agentic/reports` | List all report files | `{ count, files[] }` |
| `GET` | `/api/v1/agentic/reports/{id}/{file}` | Download chart/report | File binary (PNG/MD) |
| `DELETE` | `/api/v1/agentic/analysis/{id}` | Delete analysis + files | `{ status: "deleted" }` |

---

# Summary

`api_endpoint.py` is the **REST API gateway** that bridges the web frontend to the AI analysis backend. It:

1. **Receives** user questions via POST
2. **Orchestrates** a multi-agent LangGraph pipeline in the background
3. **Tracks** analysis progress in memory
4. **Serves** generated charts and reports to the frontend
5. **Cleans up** when analyses are deleted

It doesn't analyze data itself — it coordinates between the frontend, the LangGraph agents, and the MCP tools that do the actual work.